АВОР

Agent Build & Operations Platform

Руководство разработчика

Как добавить в АВОР своё: данные, навык, проверку, агента и целую отраслевую вертикаль. Что в платформе данные, что код, и где между ними граница.

версия платформы **1.1** | предикатов **10** | навыков **63** | дата **10.10.2026**

> Документ описывает то, что действительно работает, и отдельно называет то, чего в платформе нет. Там, где названо число, оно сверяется с кодом тестом: если расходится — виноват документ, и его правят.

СОДЕРЖАНИЕ

1 Для кого этот документ

2 Что данные, а что код

3 Data Plane: от источника до сущности

4 Рецепт: как написать и отладить

5 Навык: файл, шаблон, источники

6 Предикаты и правила

7 Правила в интерфейсе

8 Агент и его конверт

9 Вертикаль: пакет поставки

10 Арендаторы: изоляция данных

11 Приёмка: чем проверяется работа

12 Приложение А. Библиотека предикатов

13 Приложение Б. Формат пакета вертикали

14 Приложение В. Где что лежит

РАЗДЕЛ 1

Для кого этот документ

Для того, кто расширяет платформу: подключает данные заказчика, пишет навыки, добавляет проверки и собирает отраслевые вертикали.

1.1 Три роли и что каждой можно

РОЛЬ	ЧТО ДЕЛАЕТ	ЧЕМ ОГРАНИЧЕНА
Методолог заказчика	Правила проверок, справочники, сопоставление категорий, карта процессов	Работает в интерфейсе. Кода не пишет и не может
Партнёр-внедренец	Рецепты данных, навыки, агенты, пакет вертикали	Пакет вертикали **не несёт** Python: в периметре с данными заказчика чужой код не исполняется
Разработчик вендора	Всё перечисленное плюс предикаты, адаптеры источников, ядро	Релиз с тестами и приёмкой на исторических данных

ГЛАВНОЕ СЛЕДСТВИЕ

Потолок самостоятельности заказчика и партнёра — **библиотека предикатов**. Правила они пишут сами, новые *формы* проверки — нет. Это не ограничение лицензии, а условие безопасности: загрузка стороннего кода в контур с данными заказчика откроется только вместе с песочницей, и это будет отдельное решение, а не побочный эффект формата пакета.

1.2 Чего в этом документе нет

- **Установки и настройки среды** — это «Руководство администратора»: переменные окружения, авторизация, лимиты, наблюдаемость, резервное копирование.
- **Работы с платформой каждый день** — это «Руководство пользователя».

- **Обещаний на будущее.** Разделы заканчиваются подразделом «чего пока нет», и там перечислено то, что обнаружится при внедрении, если о нём не сказать заранее.

РАЗДЕЛ 2

Что данные, а что код

Главное архитектурное решение платформы. Всё остальное из него следует.

2.1 Граница

ДАННЫМИ	КОДОМ
Что проверяем и по каким правилам; какие поля обязательны; какие источники откуда читать; из каких шагов состоит агент; что считается нарушением и как это назвать человеку	Как именно считается нарушение; как обходится граф связей документов; как агрегируются записи; как нормализуется кривой ввод; сколько стоил прогон

Отсюда правило, которое проще запомнить формулировкой: **находки считает код, модель объясняет**. Если ответ должен выдержать «докажи» — его считает код с тестами. Если ответ это гипотеза, черновик, сведение или перевод — там работает модель.

ЗАЧЕМ ТАК

«Почему карточку развернули» и «почему реализация без счёта-фактуры» читают налоговая, аудитор и поставщик. Эти ответы обязаны быть одинаковыми в понедельник и в пятницу. Модель, дающая 97 % одинаковых ответов, здесь не годится — и дело даже не в цене: 140 карточек × 7 проверок это 980 вызовов модели против нуля.

2.2 Где модель уместна

РАБОТА	ПОЧЕМУ НЕ ПРАВИЛО
Найти правила, которых нет	Витрина развернула 68 карточек, правила объясняют 44. Остальные 24 — требования, которые никто не публиковал. Модель кластеризует необъяснённый остаток и предлагает кандидатов
Нормализация входа	Регламент в PDF, комментарий модератора свободным текстом, «объём 300 мм», ООО «Ромашка» ≠ Ромашка ООО. Предикат умеет сравнивать, но не разбирать
Объяснение и действие	Находка посчитана — письмо поставщику, объяснение бухгалтеру, сведение 300 находок в три системные причины пишет модель
Навигация	Агент решает, что тянуть, в каком порядке, какой навык звать, когда отдать человеку
Длинный хвост вопросов	«Что странного у этого контрагента?» — нужно суждение, а не вердикт. Правило на такой вопрос не напишешь

2.3 Шесть слоёв расширения

Снизу вверх, от самого частого к самому редкому:

1. **Правило**
   Предикат с параметрами. Методолог, минуты, без релиза.

2. **Рецепт**
   «Источник → каноническая сущность». Внедренец, минуты, без релиза.

3. **Навык**
   Методика одного шага плюс схема ответа. Внедренец, релиз пакета.

4. **Агент**
   Граф навыков и конверт прав. Внедренец, без релиза платформы.

5. **Вертикаль**
   Пакет: правила, рецепты, навыки, контракт, агент. Одна установка.

6

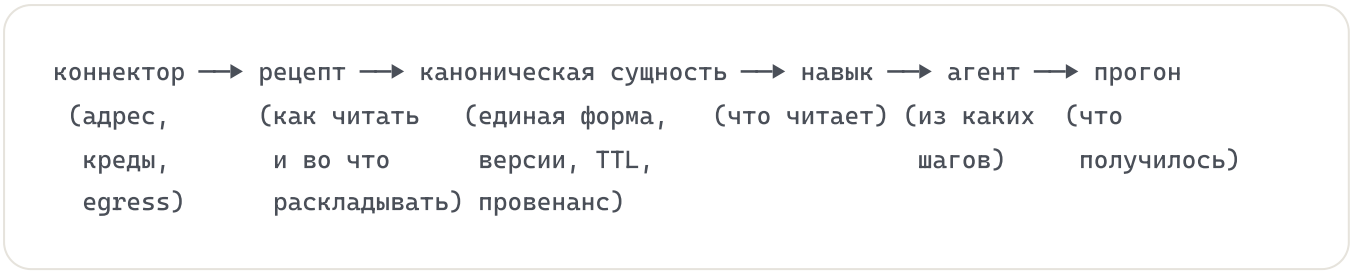
Предикат или адаптер

Новая форма проверки или новый вид источника. Вендор, релиз с тестами.

РАЗДЕЛ 3

Data Plane: от источника до сущности

Данные попадают в платформу одной дорогой: коннектор знает, где лежит источник; рецепт знает, как превратить его строки в каноническую сущность.



3.1 Коннектор

Инстанс источника: вид адаптера, адрес, креды, признак выхода наружу. Заводится в интерфейсе («Данные → Коннекторы») или ручкой `POST /api/data/connectors`.

Система из реестра важнее адреса. Если коннектор или рецепт привязан к системе (`system_id`), адрес и `egress` берутся из реестра систем, а права агента на эту систему проверяются при прогоне. Рецепт без системы ходит по адресу, вписанному руками: реестр о нём не знает, и проверить права по такому адресу нечем.

3.2 Каноническая сущность

Единая форма записи, на которую смотрят навыки. Описана в коде (`CANONICAL_SCHEMAS`): обязательные поля, ключи соединения, подсказка схемы. Добавление сущности — работа вендора, потому что на неё смотрят навыки и проверки.

У каждой записи есть три вещи, которых нет в исходном источнике:

ПОЛЕ	ЧТО ЭТО	ЗАЧЕМ
`provenance`	Источник, документ, время получения	Чтобы в находке была ссылка на первоисточник, а не «по данным системы»
`ttl_sec`	Срок жизни версии	Вытесняются только **перекрытые** версии; последняя версия не протухает по одному лишь времени
`schema` / `schema_version`	По какой схеме разложено	Чтобы смена схемы не ломала старые записи молча

ВЕРСИОННОСТЬ, КОТОРУЮ ЛЕГКО СЛОМАТЬ

Чтение сущности отдаёт **последнюю созданную** версию каждой записи, а не последнюю записанную. Если рецепт перечитывает источник и ставит записям одинаковое время получения, порядок определится порядком добавления — и «свежим» окажется не то, что ожидали. Время получения должно приходить из источника, а не из `now()`.

РАЗДЕЛ 4

Рецепт: как написать и отладить

Рецепт — объявление, а не программа: откуда читать, как разложить поля, что отбросить, под какой схемой записать.

4.1 Форма

```
{
  "recipe": "omninet_feed",          // имя; им же он виден в карте данных
  "version": 1,
  "status": "опубликован",
  "source":  { "kind": "json", "path": "~/.ape/files/omninet/feed.json" },
  "entity":  "feed_card",            // каноническая сущность на выходе
  "system_id": "minio",              // система реестра: адрес и egress оттуда
  "map": {
    "id":        { "col": "sku" },
    "название":  { "col": "title" },
    "категория_поставщика": { "col": "category" },
    "цена":      { "col": "price", "cast": "number" },
    "бренд":     { "lookup": "brands", "take": "name" }
  },
  "rules": [ { "assert": "id" }, { "pii_mask": ["почта", "телефон"] } ],
  "emit":  { "schema": "feed_card", "schema_version": "1.0", "ttl_sec": 604800 }
}
```

4.2 Что делает каждый блок

БЛОК	НАЗНАЧЕНИЕ
`source`	Вид адаптера и адрес. Адаптеры: csv, json, http, sqlite, postgres, xlsx, pdf, text, vector, mailpit, audit1c, computed, slava
`map`	Поле сущности ← колонка источника. `cast` приводит тип, `lookup` разыменовывает ссылку через другую сущность, `const` ставит постоянное значение
`rules`	`assert` — строка без этого поля отбрасывается (и это видно в счётчике, а не молча); `pii_mask` — маска личных данных до записи
`emit`	Под какой схемой и с каким сроком жизни писать

4.3 Отладка

1 **Разведка источника**

«Данные → Рецепты → поля источника»: платформа читает первые строки и показывает, какие колонки реально есть. Клик по колонке добавляет её в `map` — так опечатки в именах не доживают до прогона.

2 **Предпросмотр**

`POST /api/data/recipe/preview` прогоняет рецепт на выборке и возвращает канонические записи плюс счётчики: записано, отброшено, по какой причине.

3 **Прогон**

`POST /api/data/recipes/{name}/run` — запись в Data Plane. В ответе: сколько записано и сколько отброшено.

4 **Проверка в карте**

«Данные → Карта»: по сущности видно, какие рецепты её наполняют и какие навыки читают. Сущность без рецепта и навык без источника видны сразу.

ЧАСТАЯ ОШИБКА

Отброшенные строки — это результат, а не шум. Если `assert` отбросил половину фида, рецепт написан неверно или источник другой, чем ожидали. Счётчик «отброшено» нужно читать каждый раз, а не только когда «что-то не так».

4.4 Чего пока нет

- **Инкрементального чтения по курсору.** Рецепт читает источник целиком; большие таблицы режутся лимитом на стороне источника (параметры запроса, выгрузка по периоду).
- **Преобразований между полями.** `map` — соответствие один к одному плюс приведение типа. Вычисление из двух колонок делается на стороне источника или адаптером `computed`.

РАЗДЕЛ 5

Навык: файл, шаблон, источники

Навык — методика одного шага: что сделать, на каких данных, что считать результатом. Из навыков собираются агенты.

5.1 Из чего состоит

ЧАСТЬ	ГДЕ ЛЕЖИТ	ЧТО ЗАДАЁТ
Карточка	`skills/<id>/SKILL.md`	Когда применять, метод, определение готовности, антипаттерны
Шаблон извлечения	`skills/<id>/template.json` → БД	Строгая схема ответа: модель раскладывает данные по ней и не выдумывает поля
Governance-поля	`SKILL_SAFETY` в коде	`mode` (read/write/action), `egress` (internal/external), `cite`, формат вывода
Источники данных	`SKILL_DATASOURCES` + правка в UI	Какие сущности навык читает и какие поля ему нужны

СЛОВАРЬ GOVERNANCE-ПОЛЕЙ ЗАКРЫТ

`mode` — только `read`, `write`, `action`; `egress` — только `internal`, `external`. Значение «рядом по смыслу» недопустимо: чип безопасности рисуется по ключу, и однажды приехавший `mode: "create"` уронил веб-клиент целиком — ключа не нашлось, и падение произошло в корневом рендере, то есть на всех экранах сразу. Словарь объявлен одним местом и сверяется тестом.

5.2 Порядок добавления

1 **Карточка**

Написать `SKILL.md`: когда применять, метод по шагам, что считать готовым, чего не делать. Текст читает и человек, и модель.

2 **Схема ответа**

Описать `template.json`: поля результата, обязательность, контракт входа и выхода (`inputs` — объект `{required, optional}`, `produces` — список `{path, key, join}`).

3 **Безопасность**

Объявить `mode`, `egress`, `cite`. Правило простое: пишет наружу — `action` и только через подтверждение человека; считает числа — `cite: true`.

4 **Источники**

Перечислить сущности и поля. Без этого навык на прогоне получит пустой контекст и честно скажет «нет данных» — но смысла в нём не будет.

5 **Семья**

Привязать к семье: от неё зависит видимость по отделу и то, в каких конструкторах агентов навык предложат.

6 **Тест**

Прогнать навык на демо-данных и проверить, что схема ответа выполняется, а числа совпадают с источником.

5.3 Чего пока нет

- **Пакет вертикали навыки не везёт.** Карточка — markdown и безопасна, но навык тянет за собой шаблон извлечения, источники и места в семьях. Пока это поставка вендора.
- **Своего кода в навыке нет.** Навык — методика и схема; вычисления делают предикаты, расчёты — объявленный набор операций.

РАЗДЕЛ 6

Предикаты и правила

Правило — это предикат плюс параметры, и больше ничего. Предикаты пишет вендор, правила — методолог заказчика.

6.1 Три уровня

УРОВЕНЬ	КТО ДЕЛАЕТ	ГДЕ ЖИВЁТ	ЦЕНА ИЗМЕНЕНИЯ
Правило	методолог	данные (набор правил)	минуты, без релиза
Составное правило	методолог	данные	минуты
Новый предикат	разработчик	`cli/checks.py`	релиз с тестами

Как только в параметрах правила заводится арифметика, цикл или условие на условии — получается язык программирования без отладчика, типов и тестов, который умеем читать только мы. Это хуже обычного Python.

6.2 Три маркера «здесь нужен предикат»

Если попало под любой — правило не собирается:

1. вычисление по **множеству** записей (агрегация, сверка двух наборов);
2. обход **графа связей** (цепочка документов-оснований);
3. нетривиальная **нормализация** (нечёткое сопоставление, разбор текста).

6.3 Как устроено правило

```
{
  "код": "ATTR_MISSING",
  "предикат": "поле_обязательно",
  "применять_если": { "тип": ["РеализацияТоваровУслуг"] },   // фильтр записи
  "параметры": { "поля": "@правила.обязательные_атрибуты", "в": "атрибуты" },
  "причина": "не заполнен обязательный атрибут категории",
  "серьёзность": "высокая",
  "чинится": "из данных",
  "подсказка": "запросить значения у поставщика"
}
```

Предикат не знает о вертикали. Он отвечает только «нарушено или нет» и доказательством. Код находки, причина, серьёзность и «чинится» приходят из правила — поэтому один предикат служит и карточкам товара, и бухгалтерским документам.

6.4 Ссылки: ровно две формы

```
$запись.поле      значение поля проверяемой записи
@имя.поле         поле справочника, связанного в блоке «контекст» набора
```

Расширять набор форм — только через ADR. Третья форма это начало языка.

6.5 Контекст: как правило дотягивается до справочника

```
"контекст": [
  { "имя": "категория", "из": "category_map",
    "по": { "категория_поставщика": "$запись.категория_поставщика" },
    "взять": "категория_витрины",
    "если_нет": { "код": "CATEGORY_UNMAPPED", "чинится": "человеком",
                  "причина": "категория поставщика не сопоставлена категории
витрины" } },
  { "имя": "правила", "из": "showcase_rule", "по": { "категория": "@категория" } }
]
```

Связь, которая ничего не нашла, — **не техническая ошибка, а находка по существу**. Либо она объявлена в `если_нет` (и тогда это предметная находка), либо движок сам вернёт `КОНТЕКСТ_НЕ_РАЗРЕШЁН`. Молчания быть не должно: правило, которому нечем параметризоваться, просто не сработает, и запись окажется «чистой».

ТАК ЭТО ВЫГЛЯДЕЛО НА ПРАКТИКЕ

В наборе стояло `$категория` вместо `@категория`. Ссылка разрешилась в саму себя, справочник не нашёлся, шесть правил молча не сработали — полнота проверок упала с 74 % до 20 %, и все карточки выглядели исправными. Отсюда две защиты: громкая находка в прогоне и статическая проверка набора при сохранении. Если связь действительно необязательна — пишется `"необязательно": true`, осознанно и видно.

6.6 Цепочка находки

То, что человек читает в карточке расследования, описывается метками в правиле:

```
"цепочка": [
  { "звено": "Реализация", "документ": "$запись" },
  { "звено": "Счёт-фактура выданный (НДС)", "есть": false, "примечание": "счёта-
фактуры нет" },
  { "звено": "Оплата · взаиморасчёты", "документ": "@оплата",
    "примечание_если_нет": "поступления на р/с нет" }
]
```

`$факт.<ключ>` в примечании подставляет значение, которое вернул предикат. Иначе каждая новая проверка снова требовала бы Python ради трёх подписей.

6.7 Как добавить предикат

1 **Проверьте, что он нужен**

Предикат добавляется, когда его просят **два разных заказчика** или он закрывает **пять и больше правил** в одной вертикали. Иначе библиотека превращается в свалку из двухсот блоков, где нужный не находится.

2 **Напишите функцию**

`(запись, параметры, контекст, наборы) → list[{поле, доказательство, факты}]`; агрегатный — `(записи, параметры, наборы)` и помечается `по_набору`. Никаких обращений к хранилищу: данные приходят аргументами, иначе предикат нельзя прогнать на исторической выгрузке.

3 **Объявите в реестре**

`применить`, `параметры` (по ним редактор строит форму), `описание`.

4 **Доказательство — обязательно и конкретно**

Не «нарушено правило», а «2 при минимуме 3», «„300 мм“ — ожидается единица „л“». Находка без доказательства неаудируема.

5 **Тесты**

Срабатывание, несрабатывание, пустые и битые данные. Предикат не имеет права падать на кривой записи: прогон не должен умирать из-за одной строки фида.

6 **Приёмка на исторических данных**

Где есть метка (лог отказов, слепая разметка эксперта) — прогон по прошлому периоду: сколько сработал, сколько совпало, сколько ложных. Не совпал — не принят.

6.8 Здоровье библиотеки

ОДНА МЕТРИКА

Сколько новых предикатов потребовала очередная вертикаль. Первая потребовала шесть, вторая — четыре, в библиотеке сейчас десять. Третья должна потребовать ноль-один. Если снова четыре — библиотека не работает, и это сигнал пересмотреть набор, а не дописать ещё пять.

РАЗДЕЛ 7

Правила в интерфейсе

«Данные → Правила»: набор правил приезжает с пакетом вертикали, правка методолога ложится поверх отдельной записью.

7.1 Поставка и правка

СЛОЙ	ГДЕ ЛЕЖИТ	СВОЙСТВО
Поставочный набор	`verticals/<id>/rules/*.json`	Неизменяем. К нему возвращает «вернуть версию пакета»
Правка заказчика	таблица `check_rules`	Переживает перенакат образа; видно, кто и когда

Отсюда три свойства, которых не было бы при правке файла: «вернуть версию пакета» возможно всегда; правка не исчезает при обновлении; набор, по которому считались находки, **атрибутируем** — аудитору отвечают «правило изменил такой-то такого числа», а не «правила изменились».

7.2 Что делает редактор

- **Форма параметров строится по схеме предиката**, которую отдаёт сервер (`GET /api/data/predicates`). Список, переписанный во фронт руками, разошёлся бы с библиотекой через месяц: предикат добавили, строку забыли.
- **Смена предиката обнуляет параметры.** У каждого предиката своя схема, и чужие значения пройдут валидацию набора, но правило по ним молча не сработает.
- **Сохранение валидирует набор и отказывает.** Неизвестный предикат или неразрешимая ссылка на справочник означают, что правила не сработают, а записи окажутся «чистыми». Поэтому отказ в сохранении, а не предупреждение в углу.

7.3 Прогон на выборке до сохранения

`POST /api/data/rules/{id}/dryrun` считает находки по **несохранённому** набору на двадцати записях. В ответе не только находки, но и счёт по каждому правилу — включая нули.

> **ПОЧЕМУ НУЛИ ПОКАЗЫВАЮТСЯ**
>
> Нуль у правила — такой же результат, как сотня: он отвечает на вопрос «моё правило вообще срабатывает?». Пустая графа на этот вопрос не отвечает.

Отдельно от находок показываются две графы:

ГРАФА	ЧТО ОЗНАЧАЕТ
Из блока «контекст»	Связь не нашлась, и набор это объявил (`если_нет`). Находка по существу — правило сработало как написано
Набор сломан	`КОНТЕКСТ_НЕ_РАЗРЕШЁН` и прочее: связь не объявлена, правила по набору не сработают. Красная графа

Складывать их в одну значило бы либо пугать ложной тревогой, либо прятать настоящую поломку.

РАЗДЕЛ 8

Агент и его конверт

Агент — граф навыков и объявленные границы. Границы проверяются при сборке, а не при аварии.

8.1 Четыре границы конверта

ГРАНИЦА	ЧТО ОГРАНИЧИВАЕТ
Автономия A0–A4	Что агент делает сам, а что через подтверждение человека. Потолок считается по графу: узел наружу поднимает требование
Данные	Какие сущности и поля агент читает — из объявленных источников навыков
Системы	Куда он имеет право обратиться. Разрешения проверяются по реестру систем, отказ пишется в журнал
Выход наружу	Любая отправка — явно объявленным узлом и под подтверждение

8.2 Как собрать

1. **Контракт**

   `contract.json` проходит ingress: объявленные навыки, семья, потолок автономии, пакет свидетельств. Ссылка на свидетельство — `ev:<10 hex>`, совпадающая с началом записи в пакете.

2. **Граф**

   `agent_body.json`: узлы-навыки и связи. Проверка сборки сверяет граф с конвертом и отказывает, если граф просит больше объявленного.

3. **Версия**

   Сохранение правит последний черновик, а не плодит версии. Новая версия — осознанным «Пересмотром».

4 Посадка

Агент становится рабочим, когда посажен на участок карты процессов. Установка пакета вертикали этого не делает: решение человека.

ЧТО ПРОВЕРЯТЬ ГЛАЗАМИ

Верификация конверта пересчитывается на каждое сохранение графа: граф изменился — прежний вердикт относится уже не к нему. Если в карточке агента вердикт выглядит «старым», значит граф правили после него.

РАЗДЕЛ 9

Вертикаль: пакет поставки

Вертикаль — единица поставки: то, что устанавливают одной операцией и за что платят. Не папка по соглашению.

9.1 Раскладка

```
verticals/<id>/
  vertical.json        манифест: что это, версия, что содержит, что требует, чем мерить
  contract.json        конверт прав агента (проходит ingress)
  agent_body.json      граф агента: узлы-навыки, связи, автономия
  rules/*.json         наборы правил
  recipes/*.json       рецепты «источник → каноническая сущность»
  norms/*.md           (опц.) экспертный справочник норм для вердикта
```

9.2 Манифест не имеет права врать

Сверка идёт **в обе стороны**: что объявлено в `содержит`, обязано лежать в папке, и наоборот — файл, который есть, но не объявлен, тоже ошибка. Причина простая: пакет читают глазами по манифесту, а ставят по файлам. Если они разошлись, заказчик получит не то, что прочитал в описании, и узнает об этом на прогоне.

Плюс при сверке прогоняется статическая проверка каждого набора правил. **Невалидный пакет не ставится** — не «ставится с предупреждением».

9.3 Поле «метрика» обязательно

Пакет обязан сказать, **чем** мерить его работу на данных заказчика и **почему** именно этим. Пилот продаётся как измерение на прошлом периоде, а не как обещание процента. Вертикаль без метрики нельзя ни принять, ни защитить — это проверяет тест.

9.4 Установка

```
POST /api/verticals/<id>/install    {"перезаписать_правки": false}
```

ШАГ	ЧТО ДЕЛАЕТ	ПОВТОРНАЯ УСТАНОВКА
контракт	ingress-валидация и приём	уже принят → пропущен
правила	поставочный набор читается из пакета	есть правка методолога → **пропущен**
рецепты	нормализация и запись в Data Plane	перезапись, рецепт один на имя
агент	черновик из `agent_body.json`	версии не плодятся

Каждый шаг отчитывается словом **поставлен / обновлён / пропущен** с причиной. Молча установка не делает ничего: заказчик должен видеть, что именно изменилось в его системе.

> ГЛАВНОЕ ОБЕЩАНИЕ ФОРМАТА
>
> **Правку методолога установка не трогает.** Заказчик, который правил правило в интерфейсе, не должен потерять свою работу от того, что вендор выпустил новую версию пакета. В отчёте так и написано: «в системе есть правка v3 (buh.analyst), установка её не трогает». Снять правку можно только явным `перезаписать_правки: true`.

9.5 Из командной строки

```
export ABOP_API=https://abop.example.com   # адрес сервера
export ABOP_TOKEN=<JWT>                    # токен manager+
ape verticals                              # что есть и что уже стоит
ape verticals omninet                      # поставить
ape verticals omninet --overwrite          # поставить, сняв правки методолога
```

CLI ходит **в ту же ручку**, что интерфейс. Второго пути установки быть не должно: он разойдётся с проверками — сверкой манифеста, требуемыми навыками, бережным отношением к правкам. Тест запрещает в команде прямые обращения к хранилищам.

9.6 Чего пока нет

- **Навыки пакет не везёт** (см. 5.3).

- **Карта процессов в пакет не входит.** Агент ставится черновиком, участок карты заказчик создаёт сам. Для первой вертикали это нормально, для десятой — повод вернуться к формату.
- **Нет версионного обновления.** Установка новой версии поверх старой не считает разницу и не предлагает миграцию: рецепты перезаписываются, правила остаются.

РАЗДЕЛ 10

Арендаторы: изоляция данных

На платформе живут несколько компаний. Изоляцию обеспечивают хранилища, а не маршруты.

10.1 Правило, из которого всё следует

Если проверять арендатора в ручках, то каждая новая ручка — новая дыра, и найдётся она у заказчика. Поэтому арендатор живёт в контексте запроса и ставится один раз, в шве аутентификации, а фильтруют по нему **хранилища**: ручка физически не может прочитать чужое, даже если её автор о тенантах не думал.

10.2 Откуда берётся

Из токена — заявка `tenant` (или `org`). Это **не выбор пользователя**: арендатор у человека один, как и работодатель. Мусор в заявке отбрасывается явно, а не превращается в «похожий» идентификатор.

Вендорская поддержка (`admin` / `support`) может открыть данные арендатора заголовком `X-ABOP-Tenant`. Три условия, и все обязательные: разрешает сервер, а не интерфейс; переход пишется в append-only журнал; в интерфейсе висит **постоянная плашка** — не разовый тост, иначе вендор примет чужие находки за свои.

10.3 Арендатор в первичных ключах

Не только в фильтре — потому что идентификаторы у разных арендаторов **совпадают по построению**. Поставили одну вертикаль — контракт у обоих `omninet-showcase-2026`, агент `…v1`, набор правил `showcase-b`, рецепт `omninet_feed`. Пока ключ был одиночным, второй заказчик затирал первого — молча и необратимо.

10.4 Два правила для того, кто пишет код

1. **Фильтр пишется текстом запроса**

   А не первым элементом списка условий: список можно переставить, и статическая проверка этого не увидит. Условие, которое нельзя потерять, должно быть видно глазами.

2

Инъекция в ядро — читателем, а не снимком

Ядро синхронно, и часть состояния сервер в него инжектирует. Готовый список (рецепты, правки навыков) про арендатора не знает по определению: вендор, открывший данные клиента, видел рецепты своего. Поэтому инжектируется функция, читающая по месту.

ЕЩЁ ОДНА ЛОВУШКА, СТОИВШАЯ ОТЛАДКИ

Зависимость, которая ставит арендатора, обязана быть **асинхронной**. FastAPI исполняет синхронную зависимость в пуле потоков: контекст туда копируется, но присваивание обратно в задачу запроса не возвращается. Пока зависимость была синхронной, `/api/me` честно показывал переключение, а хранилища читали арендатора по умолчанию — изоляция не работала вовсе.

10.5 Фоновые пути

Прогон исполняет **воркер**, а не запрос. Арендатор едет в самом задании и ставится воркером перед исполнением. **Планировщик** — вторая такая точка, и ошибка там тише: если просто читать список агентов, он увидит только арендатора по умолчанию, и расписания остальных компаний молча не запустятся. Поэтому планировщик обходит арендаторов по списку.

Законных сквозных читателей ровно два: взятие задания из очереди (воркер общий для всех компаний) и перечень арендаторов для планировщика. Всё остальное сквозным быть не имеет права.

10.6 Чего пока нет

- **Реестра арендаторов.** Арендатор — строка из токена; списка «какие компании есть, с каким тарифом и лимитами» не существует. Нужен раньше, чем второй платящий клиент.
- **Квот по арендатору.** Потолок токенов и бюджет общие на установку.
- **Заявки `tenant` в каталоге пользователей.** До её настройки все попадают в арендатора по умолчанию — это и делает миграцию безопасной.
- **Переноса данных между арендаторами.** «Перевести клиента с пилотного стенда в его арендатора» — ручная операция, инструмента нет.

РАЗДЕЛ 11

Приёмка: чем проверяется работа

Приёмка расширения — не «работает», а «совпадает» и «измерено». Ниже четыре приёма, которые на этой платформе себя оправдали.

11.1 Базовая линия при переезде

Когда проверки вертикали переезжают с собственного кода на общий движок, приёмка — совпадение находок **поле в поле** с линией, снятой ДО переезда. Любое расхождение — регресс, даже если новая находка выглядит разумнее: аудиторский вывод не должен меняться от того, что мы переставили код.

> ЧТО ЭТО ДАЁТ НА ПРАКТИКЕ
>
> Сверка аудита 1С по всем полям, включая цепочки, поймала три дефекта, которые «работает» пропустило бы: в цепочку попадала вся запись вместо короткой ссылки; подстановка фактов съедала начало длинного ключа; ссылка на связь без поля не подставлялась вовсе — в находку уходил сам текст ссылки.

11.2 Измерение на исторических данных

Там, где есть фактическая разметка (лог отказов витрины, заключения эксперта), новое правило или предикат прогоняется по прошлому периоду: сколько сработал, сколько совпало с фактом, сколько ложных, и **сколько фактов осталось необъяснёнными**. Последнее число — не позор, а следующая работа: это требования, которых нет ни в одном регламенте.

11.3 Статический сторож там, где поведенческий тест слеп

Поведенческий тест проверяет то, о чём автор подумал. Забытое им проверяет статическая проверка исходника. На платформе их несколько, и каждая появилась после реальной поломки:

СТОРОЖ	ЧТО ЛОВИТ
Снимок маршрутов	Ручка исчезла при переносе между модулями
Разбор имён в AST	Функция переехала, а звавший её код остался: Python обнаружил бы это только на живом запросе
Словарь governance-полей	Значение «рядом по смыслу» (`mode: "create"`) в карточке навыка
Фильтр арендатора в SQL	Запрос к таблице с арендатором без фильтра по нему — то есть утечка
Сверка таблиц в документах	Предикат добавили, строку в руководстве забыли

СТОРОЖ, КОТОРЫЙ НИЧЕГО НЕ РАЗОБРАЛ, ХУЖЕ ОТСУТСТВУЮЩЕГО

Первая версия проверки SQL искала литералы регуляркой по кавычкам. Один апостроф в русской строке документации «съел» следующие тысячи символов: проверка находила НОЛЬ запросов и была зелёной. Разбор переведён на `ast`. Всегда проверяйте, что сторож действительно что-то видит, — например, внесением заведомой дыры.

11.4 Тест проверяет результат, а не наличие кода

Запись о переходе вендора на данные клиента первое время не велась вообще: вызов журнала падал на пересечении имён аргументов, а `except Exception: pass` это скрывал. Тест, смотрящий на исходник, остался бы зелёным.

Отсюда два правила. Проверяйте **запись**, а не строку кода, которая должна её сделать. И если глушитель закрывает «некритичный путь» — сужайте его до конкретного отказа, иначе он закроет и опечатку.

11.5 Что запускать перед сдачей

```
python -m pytest -q                    # весь набор
python webapp/build.py check           # бандл собран из исходников
python webapp/build.py lint            # синтаксис JS шаблона и компонентов
```

ПРИЛОЖЕНИЕ А

Библиотека предикатов

Десять форм проверки, которых хватило на две отрасли. Таблица сверяется с кодом тестом: предикат добавили — строка обязана появиться.

ПРЕДИКАТ	ЧТО ПРОВЕРЯЕТ	ПАРАМЕТРЫ
`поле_обязательно`	обязательные поля заполнены	`поля` · `в` · `при`
`единицы_измерения`	значение атрибута содержит ожидаемую единицу измерения	`единицы` · `в`
`не_меньше`	числовое поле не меньше минимума	`поле` · `минимум`
`не_длиннее`	текстовое поле не длиннее предела	`поле` · `предел`
`не_содержит`	в поле нет запрещённых подстрок	`поле` · `запреты`
`значение_из_словаря`	значение поля есть в допустимом списке (или наоборот — «входит»)	`поле` · `значение` · `допустимые` · `нарушение_если`
`связанный_документ_есть`	есть связанная запись; разрыв цепочки — когда её нет	`из` · `по` · `поле`
`период_совпадает`	две даты в одном периоде; переходящая операция — когда в разных	`поле` · `с` · `точность`
`есть_строка_со_значением`	во вложенном списке есть строка с нужным значением	`в` · `поле` · `равно` · `через_справочник`
`дубль_по_ключу` **(агрегатный)**	один ключ у нескольких записей — дубль справочника	`ключ` · `где` · `подпись` · `звено`

Агрегатный предикат вызывается один раз на весь набор, а не на каждую запись: «дубль по ключу» — вычисление по множеству, и по первому маркеру раздела 6.2 оно и попало в код, а не в правило.

ПРИЛОЖЕНИЕ Б

Формат пакета вертикали

Манифест `vertical.json` целиком, с пояснением каждого поля.

```
{
  "id": "omninet",                      // == имя папки, иначе отказ сверки
  "название": "Открытый протокол e-com: карточка для чужой витрины",
  "версия": "1.0.0",
  "область": "management",              // семья/отдел по умолчанию
  "описание": "...",                    // что делает — читает человек
  "автор": "вендор",
  "источник": "опубликованные требования витрины B + лог отказов за период",
  "требует": {                          // должно быть в системе ЗАРАНЕЕ
    "сущности": ["feed_card", "showcase_rule", "category_map", "showcase_reject"],
    "адаптеры": ["json"],
    "навыки":   ["feed-checks", "feed-fix", "feed-scorecard"]
  },
  "содержит": {                         // сверяется с папкой в ОБЕ стороны
    "контракт": "contract.json",
    "агент":    "agent_body.json",
    "правила":  ["showcase-b"],
    "рецепты":  ["omninet_catmap", "omninet_feed", "omninet_rejects",
"omninet_rules"],
    "нормы":    "norms"                 // необязательно
  },
  "метрика": {                          // ОБЯЗАТЕЛЬНО: чем мерить и почему
    "как":    "полнота и точность против лога отказов витрины",
    "почему": "обещать процент нельзя — он считается на прошлом периоде заказчика"
  }
}
```

Б.1 Отказы установки и что они значат

СООБЩЕНИЕ	ЧТО ДЕЛАТЬ
«манифест обещает набор правил X, файла нет»	Либо положить файл, либо убрать из `содержит`
«в пакете есть рецепт Y, в манифесте не объявлен»	Дописать в `содержит`: пакет читают по манифесту
«id манифеста не совпадает с папкой»	Переименовать папку или поправить `id`
«набор «X»: неизвестный предикат / неразрешимая ссылка»	Поправить набор; правила по нему не сработали бы молча
«в системе нет требуемого — навыки: …»	Навыки и адаптеры ставит вендор; пакет их не везёт

ПРИЛОЖЕНИЕ В

Где что лежит

Карта репозитория для того, кто пришёл первый раз.

ПУТЬ	ЧТО ВНУТРИ
`cli/ape.py`	Ядро: навыки, рецепты, канонические записи, прогон, инструменты агента. Работает и без сервера
`cli/checks.py`	Библиотека предикатов и движок проверок по наборам правил
`server/routes/*.py`	HTTP по доменам: данные, правила, вертикали, агенты, прогоны, карта, админка
`server/deps.py`	Шов: кто пользователь, уровни доступа, ABAC, арендатор
`server/tenancy.py`	Арендатор: контекст, нормализация, миграция ключей
`server/*_store.py`	Хранилища. Фильтр по арендатору — здесь, а не в маршрутах
`skills/<id>/`	Карточка навыка и шаблон извлечения
`verticals/<id>/`	Пакеты вертикалей — поставка
`demo/<id>/`	Демо-сценарии для показа. **Не** поставка
`webapp/src/`	Исходники веб-клиента; `webapp/index.html` собирается из них
`tests/`	Тесты. В начале каждого файла написано, какое правило он сторожит и почему
`docs/`	Решения (ADR), описания слоёв, эти руководства в `docs/guide/`

B.1 Где читать дальше

ТЕМА	ДОКУМЕНТ
Решения и их причины	`docs/ABOP_ADR.md`
Предикаты и правила подробно	`docs/PREDIKATY.md`
Пакет вертикали подробно	`docs/VERTIKALI.md`
Арендаторы подробно	`docs/ARENDATORY.md`
За что платит заказчик	`docs/MONETIZATION.md`
Ручки API	`docs/ABOP_API.md`

СОПУТСТВУЮЩИЕ ДОКУМЕНТЫ

«ABOP · Руководство пользователя» — как работать с платформой каждый день.
«ABOP · Руководство администратора» — как развернуть и держать под контролем.
Этот документ описывает третью сторону: как добавить в платформу своё.